Data Science Playbook for Researchers

Bevin Manuelpillai

Contents

My Philosophy of Coding as a Researcher **3**

The Motivation to Learn to Code as a Researcher . . . 3

How Scientists Should Approach Coding . . . 3

The Doors Coding Opens for Researchers . . . 3

Why Coding is Fun . . . 3

Forest vs. Trees . . . 4

Programming and Data Science Learning Path **5**

Phase 1: Getting Started . . . 5

Phase 2: Writing Cleaner Code . . . 5

Phase 3: Apply your skills to research . . . 6

Phase 4: Optimization and Customization . . . 6

Data Manipulation in R and Python **7**

Introduction to Functional Programming in R **9**

Topics Covered . . . 9

Why Use Functions . . . 9

Control Flow in R . . . 10

Loops . . . 10

Level 2: Working with Packages . . . 11

Debugging in R **12**

Version Control your Code **13**

Why you should learn Git and Github . . . 13

General Git workflow . . . 13

4 Levels of Version Control . . . 13

Getting Started with GitHub . . . 13

Working with Branches . . . 14

Advanced Git Features 16
Continuous Development using Git 16

Command Line Tools **18**
File Management 19
Package Management 19
Tips for Effective Terminal Use 19
Introduction to Shell Scripts 20

Python Environments **21**

Tools for Machine Learning in Python **23**

High Level Bayesian Inference **26**

Programmer's Toolbox **27**

Useful Resources for Programmers **29**

My Philosophy of Coding as a Researcher

Coding as a Tool for Research

The Motivation to Learn to Code as a Researcher

The Start Is The Hardest Part

When you first start to program it can feel really overwhelming. You aren't comfortable with the language, you have to look up everything before you can do what you want. You'll see more experienced programmers doing things a lot faster and in ways you've never seen before. It can be feel really overwhelming, but rest assured this will be the hardest part of your programming journey. Once you gain experience and confidence as a programmer, you will feel like you can tackle any problem.

How Scientists Should Approach Coding

Video: How to learn programming - Advice for scientists | Clara Sousa-Silva and Lex Fridman

The Doors Coding Opens for Researchers

Developing your coding skills opens a lot of doors to you when you go to analyze your data. If you have lots of data, you probably will not be able to run your analysis on your local computer due to hardware constraints. Learning some basics of using computing clusters or cloud computing will allow you to tackle this problem. Also, a lot of statistical methods such as bayesian inference require a lot of computation power.

Why Coding is Fun

Coming from a STEM background one of my favorite parts about programming is how you get instant feedback from running your code. Either your code runs or you get an error message. Every time your code runs you get a feeling of accomplishment that drives you to over come the next bug in your code. Every time you squash a bug in your code, you improve your ability to diagnose and fix the next one.

One of my favorite parts of coding is the community of developers and researchers that are there to support you with their online material. You can look at other people's projects or online tutorials for inspiration. I highly recommend looking for course websites that professors post online as they are tailored for researchers and often include homework assignments to reinforce your learning. Most importantly, you learn by doing so pick projects you enjoy doing.

Forest vs. Trees

Don't mistake the forest for the trees. I think coding is especially fun for scientists because it isn't our main job. We use code as a tool to answer the questions that interest us, but our goal is not make the most optimized or well-written. You may happen to become really good at coding as a byproduct of tackling your research questions, coding is just a mean to reach that goal. Write code first, then optimize it later if needed. Now, that is not to say that the code shouldn't be readable – it should still be well documented so that others can work on it too.

I include a lot of tools that make coding a lot easier and fun, but when you are first starting focus on getting the basics down. This comes with time and practice. I recommend following one course and then focusing most of your time on projects.

Programming and Data Science Learning Path

Four Phases Building Foundational Programming Skills

Phase 1: Getting Started

Get comfortable working with data. You should be able to manipulate a data frame easily such as filter rows, select columns, create new columns, visualize your data, and calculate basic statistics.

The next big milestone would be to get comfortable working with functions. Using functions will revolutionize how you program. A first, it may seem like writing functions takes way more time but in the long-term functions make your code more readable, less prone to bugs, and easier for you to comeback to.

- Start with this Statistical Programming Course
  - Complete the homework assignments
- Read R for Data Science
- Experiment with functions, loops, and control flow in your code
- Learn GitHub for version control
- If machine learning is needed for your research:
  - Learn basics of Pandas and NumPy in Python
  - Use Poetry to manage Python dependencies
  - Complete ML tutorials on Kaggle
  - Read through Sci-KitLearn Documentation
  - Participate in Kaggle learning competitions
  - Study how experts approach regression and classification tasks
- Consider learning SQL to efficiently manage and query large datasets

Note: Getting comfortable working with data is probably the hardest part. You will start out being uncomfortable with the syntax, having to constantly look things up, and getting frustrated. Remember this is the hardest part and you will be gaining a lot of momentum going forwards.

Phase 2: Writing Cleaner Code

- Read Hadley Wickham's Advanced R and R Packages
  - Work towards creating well-documented Functions to share with others
- Learn about non-standard evaluation for using tidyverse syntax in functions

- Develop debugging skills for functions
- Get comfortable with:
  - Git branches, rebasing, and pull requests
  - Command-line tools and terminal navigation
  - Simple bash scripting
- *(Optional)* Read "Grokking Algorithms" to understand basic data structures and algorithms
- Focus on refining your statistics skills simultaneously

Note: This is really useful if you share code with others and do a lot of data cleaning, or other repetitive coding tasks

Phase 3: Apply your skills to research

- Balance coding skills with research questions and statistical knowledge
- Explore Bayesian frameworks that require computational power
- Learn to use:
  - High-performance computing clusters (if available)
  - AWS or Azure cloud computing

Note: These 3 phases are the most important to have a strong foundation to data science. The next phase is a collection of things that I found useful in my own research.

Phase 4: Optimization and Customization

- Switch to VS Code as your editor
- Experiment with Vim motions
- Customize your terminal environment
- Watch educational videos about interesting projects
- Participate in Kaggle projects and study other people's solutions
- For advanced software development:
  - Learn Docker containers (like a box to package everything you need to run a piece of software)
  - Master cloud computing services (AWS or Azure)
- Try Advent of Code if you want to challenge yourself or learn a new language

Data Manipulation in R and Python

Get Comfortable Working with Data in R and Python

Cleaning and Visualizing Data

The best place to start working with data is to clean and visualize it. You should calculate descriptive statistics on your dataset before running any statistical tests.

Tidyverse (R)

Learning the Tidyverse is a great way to write clean, readable R code for data manipulation. It also works seamlessly with ggplot2 for data visualization.

I recommend starting with this course and this associated YouTube series by Kelsey Codes. The course quickly introduces you to Tidyverse and ggplot and provides practical exercises.

Frequently Used Tidyverse Functions

- select(): Choose specific columns
- filter(): Filter rows based on conditions
- mutate(): Create or transform variables
- arrange(): Sort data by column values
- pivot_longer(): Reshape wide data into long format
- pivot_wider(): Reshape long data into wide format
- ggplot(): Build layered plots using + to add elements

You should also look into creating Quarto or Rmarkdown files to present your code in a nice looking report

Pandas (Python)

Data manipulation in Python using pandas is conceptually similar to using base R or the Tidyverse. Once you're comfortable with one language, transitioning to the other becomes easier.

Frequently Used Pandas Functions

- df[['col1', 'col2']]: Select specific columns
- df[df['col'] == value]: Filter rows based on conditions
- df.assign(): Create or modify columns
- df.sort_values(): Sort data by column values

- df.melt(): Reshape wide data into long format
- df.pivot() or df.pivot_table(): Reshape long data into wide format
- df.plot() or use seaborn/matplotlib: Create plots (analogous to ggplot)

Introduction to Functional Programming in R

Working with Functions and Packages in R

Topics Covered

1. Purpose of Functions
2. Control Flow with If, else
3. Loops using map
4. Packages using Devtools

Why Use Functions

Q. Why use functions when you can just copy and paste your code?

A. Repeating yourself makes it more likely that you make a mistake while coding. Constantly repeating your code will make your scripts very long and hard for others to read and debug. Writing functions will also make it easier for you to reuse code that you wrote before.

Example

```
# Define function
fahr_to_kelvin <- function(temp) {
  kelvin <- ((temp - 32) * (5 / 9)) + 273.15
  return(kelvin)
}

# Call function
fahr_to_kelvin(32)
```

Advanced Function Usage

You can do more advanced things with functions, like piping one function into another:

```
fahr_to_kelvin(32) |> kelvin_to_celsius()
```

More information on Functions can be found here.

Control Flow in R

Q. What is control flow and why is it important?

A. Control flow is useful when you want certain code to run if certain conditions are met. It is typically used inside functions to change what code is executed based on the arguments that are passed in.

If...Else Control Flow Boilerplate

```
if (condition is true) {
  perform action
} else {
  # that is, if the condition is false,
  perform alternative action
}
```

If...Else Control Flow Example

```
x <- 8
if (x >= 10) {
  print("x is greater than or equal to 10")
} else if (x > 5) {
  print("x is greater than 5, but less than 10")
} else {
  print("x is less than 5")
}
```

More Information on Control Flow can be found here.

Loops

Say you have a function and want to apply it to more than one thing, what do you do? One solution is to create a loop.

Steps

1. Define the function
2. Define a vector you want to apply the function to
3. Use `map()` to apply the function to each element in the vector
4. You can also write a for loop instead of passing the function and vector into the map function

Example

```
square <- function(x) { return(x * x) }

# Create a vector of numbers
vector1 <- c(2, 4, 5, 6)

# Using map() function to generate squares
map(vector1, square)
```

More information on Loops can be found here.

Level 2: Working with Packages

Using Devtools

Working with Packages

Now you have a bunch of related functions that you keep on using in a workflow. It is very convenient to put all of these functions into a package, where you can group them together, document their function for other users, and write tests. Writing your own packages is a pretty advanced topic, but I recommend starting with Hadley Wickham's R package's book. Which covers everything from creating a package directory, writing tests, documenting functions, and publishing your package on CRAN.

Debugging in R

Squash bugs fast

Learn to Debug R code

Debugging code is one of the most important skills after you start writing your own functions. Developing the skill of debugging takes time but here is a good starting place. Some recommended reading in Hadley Wickham's Advanced R book

Debugging Functions

- **browser()**

  Put this in the body of your function, and it will pause its execution allowing you to interactively step through your function

- **debug()**

  Stops a function from executing automatically and allows you to execute one statement at a time

- **print(“”)**

  Print statements are useful to pin down where you are during a function's execution

- **VScode and RStudio Debugger**

  They have some really nice interactive debugging tools that are worth exploring

Version Control your Code

Learning Git and GitHub

Why you should learn Git and Github

Git tracks every change you make, allowing you to go back to a previous version of your code, similar to version control in word. GitHub is a cloud-based platform for hosting Git repositories, makes it simple to collaborate with others. Using features like branches, you can work on different parts of a project independently, then merge changes. Using Git you will never have to worry about your code getting deleted.

General Git workflow

```
# Start tracking Git history for a directory (folder)
git init

# Fetch and merge changes from the remote repository
git pull

# Check the status of your files
git status

# Stage the files you want to commit (you can specify files or use . for all changes)
git add .

# Commit your changes with a descriptive message
git commit -m "Describe what you changed or added"

# Push your commits to the remote repository
git push
```

4 Levels of Version Control

Getting Started with GitHub

GitHub GUI is great for 90% of what you'll be using Git for. Let's start with the basics:

- Create a GitHub account

- Download GitHub Desktop
- Download Git
- Complete the GitHub tutorial

2 Main Paths When Starting a New Project

Option 1: Clone Someone Else's Repository

To clone someone else's repository from GitHub:

- Copy the link to the repo
- Click File > Clone Repository in GitHub Desktop
- Paste the link and choose where you want to copy the Repository locally

Option 2: Make Your Own Repository

To make your own version-controlled repository:

- Click File > New Repository in GitHub Desktop
- Give your repository a name and description
- Choose a local path
- Click "Create Repository"

The main point of Git is to version control your code and collaborate with others like Google Docs.

- **Committing:** saving changes locally with messages
- **Pushing:** transferring changes to GitHub
- **Pulling:** getting others' changes locally

Learn more

Working with Branches

The next level of working in Git would be to start using branches in Git. A branch is basically a copy of a repository that you can use to try out changes to your code before adding them to your main code.

Creating and Using Branches

1. In GitHub Desktop, click "Branch" in the top menu
2. Select "New Branch" and give it a name
3. Make your changes in this branch
4. Commit the changes to the branch
5. Push the branch to GitHub

Once the changes you make to the new branch look good, you can merge those changes into your main branch. This is especially useful when you are working with other people and each want to make changes to code and then combine them together.

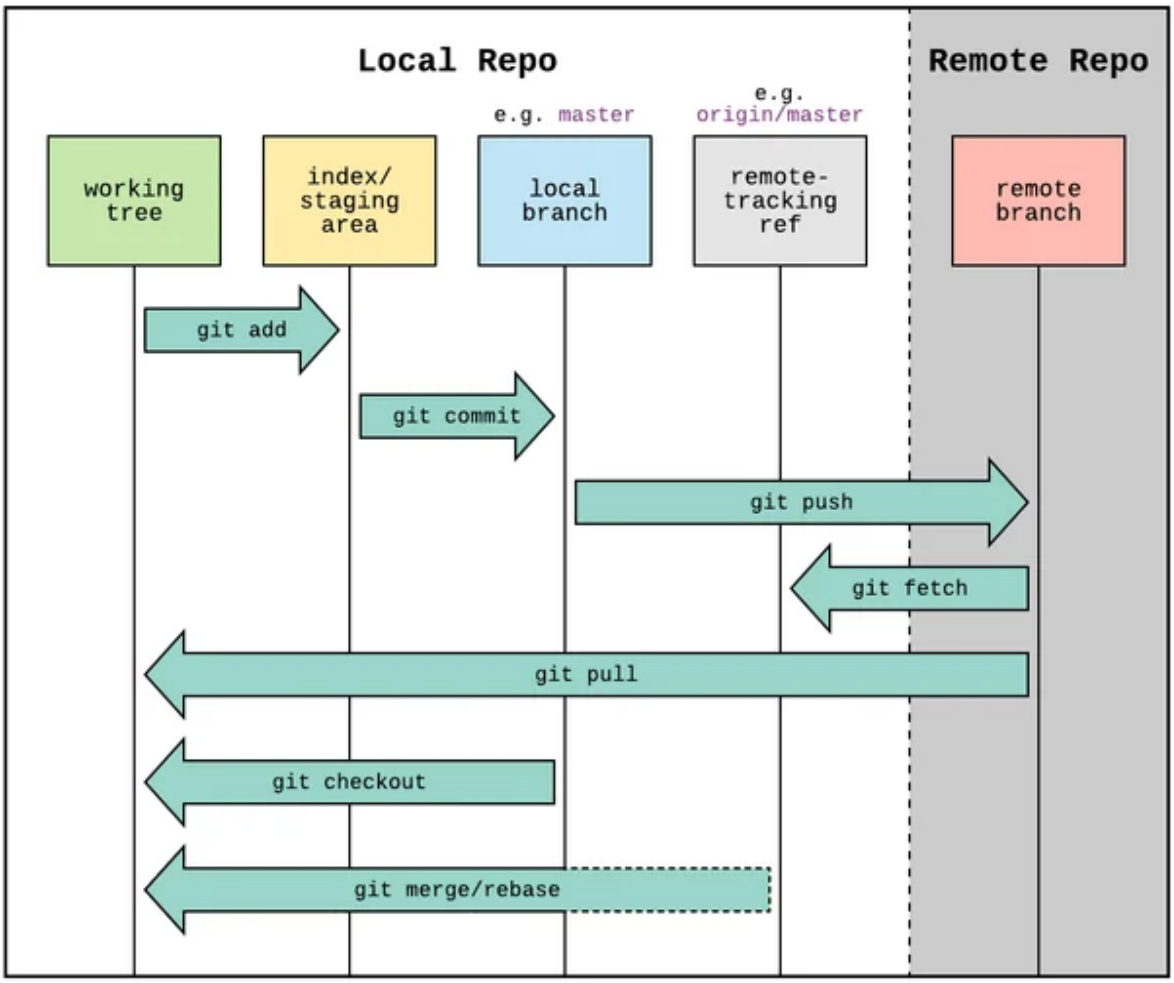


Figure 1: git workflow

Pulling changes from a feature branch into the main branch

1. Create a new branch
2. Commit changes to a branch
3. Push changes to GitHub
4. Merge a branch into main using a Pull Request

Advanced Git Features

At this point, you understand the basics of Git, and the scale of the projects you work on are becoming very large. There are some advanced tips that you should start to use to stay organized.

Advanced Features

.gitignore

A file that tells Git what files to not track. This would include data and passwords you do not want on GitHub.

More information on gitignore: Link

Rebasing vs. Merging

Rebasing is preferred over merging because it keeps a linear commit history of all the changes, so when you want to go back to a previous change, it is easier to see what change happened in what order.

More information on merging vs rebasing: Link

Squashing commits

When you want to rebase 2 branches it is super useful to squash (combined) as many commits as you can together. Rebasing works by reapplying commits on top of a new base commit, so basically takes the commits from branch B and stacks them onto branch A. This is a lot easier if you have fewer commits, and it also makes it easier for you later one when you want to look at your commit history and see what you did before.

More Information on Squashing Commits: Link

Use Git in the Terminal

Using Git in the terminal is great because you never have to leave your code editor to make changes to your code, as well as opens up a bunch of advanced features that are a bit harder using GitHub Desktop.

A useful tool for using git in the command line is lazygit allowing you to use keyboard shortcuts instead of typing out entire git command.

Continuous Development using Git

Once you start getting into the software side of coding, it is super useful to use advanced Git workflows and automation.

Professional Git Tools

Pre-commit Hooks

Use pre-commit to ensure that all code changes are consistent in style, free of common mistakes, and do not include anything unintended before they are committed to your repository.

Pre-commit is a framework for managing and maintaining multi-language pre-commit hooks. These hooks run automatically before each commit and can catch issues like formatting errors, unused imports, and more.

You need to install and configure pre-commit in each repository you want it to run in. To do this, install the tool globally with pip install pre-commit, add a .pre-commit-config.yaml file to your repository, and run pre-commit install to activate it for the repo.

Once set up, the hooks will run automatically every time you make a commit, helping enforce code quality and consistency across your project.

Pull Requests

Another advanced feature is to use pull requests in GitHub so that before anyone can add changes to your main branch, you can make sure that certain code requirements are met beforehand.

1. Create a branch with your changes
2. Push the branch to GitHub
3. Go to the repository on GitHub and click "New pull request"
4. Select your branch to merge into main
5. Add reviewers and description
6. Create the pull request

GitHub Actions

GitHub Actions are automated actions that occur after a certain event. GitHub actions go along with pull requests and are used to run tests prior to merging a branch.

Command Line Tools

Get Fast with the CLI

Why Learn to Use CLI tools

After spending some time programming you'll want to get faster at general routine tasks and you'll want to be able to easily explain what you did to someone else. I think graphical user interfaces are great when you are first learning an application, which is why I started my Version Control article with GitHub Desktop, but if you get comfortable with command line tools you can get a lot faster at your work and easy have someone reproduce what you did without having to point and click.

Getting Good with your Terminal

Learning to navigate through your computer's file system using linux commands opens up several doors for you, with the main benefit being opening you up to use

`pwd` (Print Working Directory)

This command shows your current location in the file system.

```
$ pwd
  /Users/username/Documents
```

`ls` (List)

Lists all files and directories in your current location.

```
$ ls
  Desktop  Documents  Downloads  Pictures
```

Add options like `-l` for detailed view or `-a` to show hidden files:

```
$ ls -la
```

`cd` (Change Directory)

Moves you to a different directory.

```
$ cd Documents
```

`cd ..`

Moves up one directory level (to the parent directory).

```
$ cd ..
```

File Management

`mv` (Move)

Moves files or directories to a new location, or renames them.

```
# Move a file
$ mv file.txt Documents/

# Rename a file
$ mv oldname.txt newname.txt
```

Package Management

`pip`

Package installer for Python. Use it to install Python libraries.

```
# Install a package
$ pip install numpy

# List installed packages
$ pip list
```

`brew` (Homebrew)

Package manager for macOS (and Linux). Use it to install software.

```
# Install software
$ brew install git

# Update Homebrew
$ brew update

# Upgrade installed packages
$ brew upgrade
```

Tips for Effective Terminal Use

- Tab to autocomplete (useful for completing file paths)
- Press the up arrow to recall previously commands
- Use `clear` to clear screen
- Create aliases for frequently used commands in your `.bashrc` or `.zshrc` file

Introduction to Shell Scripts

It is also useful to learn writing shell scripts which can be used to automate tasks such as managing backups, batch-renaming files, setting up development environments, automating data processing pipelines, scheduling repetitive tasks with cron, and even simplifying complex command sequences into single, reusable scripts.

I typically create bash scripts when I want to rerun R scripts with different parameter combinations on a computing cluster.

Here is a good resource on learning about bash scripts: link

Useful tools for developers

Raycast (MacOS)

- Use Shortcuts to Launch Apps
- Quickly switch between your browser and code editor

AeroSpace (MacOS)

- Tiling Manager for MacOS
- Used to easily manage windows and workspaces

Ghostty Terminal (MacOS)

- Really fast terminal
- Supports multiple tabs
- Very Customizable

Obsidian

- Markdown-based note-taking
- Great global search to quickly find notes
- Has great plugins that allow you to write formatted code

Python Environments

Using Poetry and Conda to Manage Dependencies

Going From R to Python

R is great but there are many packages that python has that R does not, especially when it comes to machine learning (scikit learn, PyTorch, etc.). One the biggest sticking points I experienced from going from R to python is managing virtual environments and dependencies. In R a lot of this is done behind the scenes and you often do not have to worry about using an explicit package version for your code. Although in Python it is important to download specific versions of your dependencies for each project. One method I really like it using poetry environments.

Poetry Installation

- Download Python onto your computer
- You want to get a package manager for Python on your computer
- Follow these instructions to get Poetry on your computer

General Workflow using Poetry

```
# set up poetry and virtual environment for your repo
poetry init
poetry lock
poetry env activate
source .venv/bin/activate
# this should create a poetry.toml file and a lock file

# Add packages to your poetry environment
poetry add
poetry update
```

Use pyenv if you need to use a different version of python than the one that is on your system.

Use Conda to Manage Dependencies for all Programming languages

Conda is similar to poetry just can be for more than just python package packages. I think both are really good and knowing a bit about both can allow you to use whatever the rest of your team is using.

```
# Create a new conda environment
conda create --name myenv python=3.9

# Activate the environment
conda activate myenv

# Install packages in your conda environment
conda install package_name

# Export environment for reproducibility
conda env export > environment.yaml

# Create environment from file
conda env create -f environment.yaml

# Update all packages in the environment
conda update --all

# Remove an environment
conda env remove --name myenv
```

Tools for Machine Learning in Python

Scikit-Learn, Tensorflow, Pytorch

Scikit-Learn

Scikit-Learn is great for traditional machine learning tasks like classification, regression, clustering, etc.

A good starting place would be getting comfortable with NumPy, Pandas, and Matplotlib in Python so that you can clean your data into features and target. After that learn the different ML algorithms and when to use them.

Tidymodels

TidyModels is similar to Scikit learn but in R and uses tidyverse principles. Read more about it here here.

Types of Machine Learning Algorithms

- **Supervised Learning**
  - Classification: Logistic Regression, Decision Trees, Support Vector Machines (SVM)
  - Regression: Linear Regression, Random Forest Regressor, Gradient Boosting
- **Unsupervised Learning**
  - Clustering: K-Means Clustering, Hierarchical Clustering
  - Dimensionality Reduction: Principal Component Analysis (PCA)
- **Reinforced Learning**
  - Large Language Models

Data Splitting

Divide your data into the following subsets:

- **Training Data:** Used for model development.
- **Validation Data:** Used for tuning model parameters and preventing overfitting.
- **Testing Data:** Used for evaluating the final model's performance on unseen data.

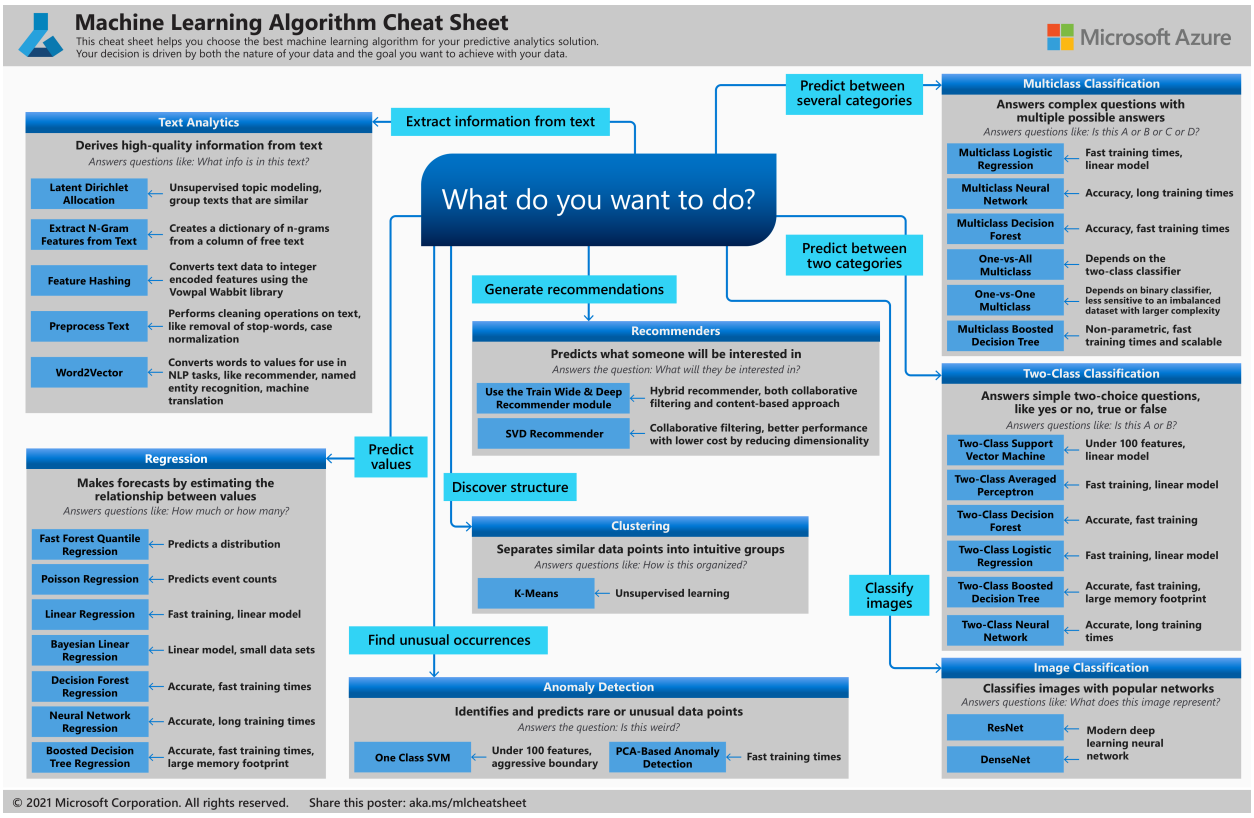


Figure 2: Machine Learning Algorithm Cheat Sheet (source)

Choose What ML Algorithm Is Right For You

Neural Networks

Neural Networks are used for deep learning and are fairly advanced. The 2 widely used deep learning frameworks are PyTorch and Tensorflow

Basic Hyperparameters in Neural Networks

- **Learning Rate:** Controls how quickly the model updates its weights.
  - Too high – the model may overshoot and fail to converge (it bounces around).
  - Too low – the model converges very slowly and may get stuck in local minima.
- **Batch Size:** Number of training examples used in one update of model weights.
  - **Stochastic Gradient Descent (SGD):** Batch size = 1
  - **Mini-batch SGD:** Batch size > 1 but smaller than the full dataset
  - **Full-batch:** Entire dataset used (rare due to memory constraints)
- **Epoch:** One complete pass through the entire training dataset.
  - Example: 1,000 samples / batch size 100 = 10 iterations per epoch

Josh Starmer has a great course on YouTube that walks you through principles of machine learning here.

High Level Bayesian Inference

Modeling Framework

Recommended Resources to Learn Bayesian Inference

Learn Theoretical Basis

This free online textbook builds up your understanding of bayesian inference from fundamental principles of probability without getting too deep into the math. I recommend starting here and then quickly applying what you learn to your own data.
A Student's Guide to Bayesian Statistics

Introduction to Stan in R

After you get a grasp of how bayesian inference works then it's time for you apply those methods to your own data. I recommend watching this video to get an introduction to using STAN in R to implement bayesian mdethods. Stan is a probability programming language which allows you to create statistical models.
Bayesian Modeling with Stan and R

Introduction to Numpyro in Python

I recommend spending sometime reading through the numpyro wiki page and then following along through the introductory tutorials.
Numpyro Wiki Page

Programmer's Toolbox

Useful Resources for Programmers

New tools

Many times switching form a tool you are comfortable with to something new can feel overwhelming. You'll feel unproductive and frustrated, but if you plan on coding in the long-term then this initial time spent will have huge benefits to you.

Why Learn VScode

VScode is the industry standard code editor, and for good reason. It is super fast, lightweight, and is highly customizable. Rstudio is great for R, but if you ever want to branch off and learn a new coding language, VScode will set you up perfectly for this. Another great addition to VScode is its integration with Copilot which can be super useful for code completions.

- Download VScode

Here is my personal VScode set up for MacOS:

- Keyboard Shortcuts with a Similar Set Up to VIM
  - Keyboard shortcuts: `files/keyboard_shortcuts_bm.JSON` in this repo
- General VScode Settings for Coding in R and Python
  - VS Code settings: `files/vscode_settings_bm.JSON` in this repo

What is VIM motions

VIM motions are a set of key bindings that let you quickly navigate and edit text without using your mouse. Basically it allows you to keep you hands on the keyboard at all times. For example with VIM motions you can quickly copy and paste a line of code, delete code inside of parenthesis, and so much more.

Why Learn VIM motions

Learn VIMotions at first is very tough, despite how intuitive the controls are. It will likely reduce your productivity for a bit until you get comfortable. One common argument that people have about why to learn VIM is that it will make you a faster programmer. I would argue that the best reason to learn VIM is that it reduces the amount of time between when an idea pops into your mind and when you fingers can start to type it out.

- VIM extension for VScode
- Learn Vim Motions
- You can also set up VIM motions in Rstudio
- I also have Vim Motions set up on my browser with the vimium chrome extension
- I recommend mapping your caps lock key to your escape key to quickly change form normal to insert mode

Autoformat Code in R

- VScode extension that autoformats your code when you save a file
  - AIR - R Language Support
- RStudio package to autoformat your code
  - Styler package Support

Useful Resources for Programmers

Useful Courses and Fun Content

Useful Courses

Name	Description
Statistical Programming in R	High-level course teaching you the basics of using Tidyverse for data analysis
Data Wrangling in Python	Teaches data wrangling in Python using Pandas
Basic Git Commands | Atlassian Git Tutorial	Comprehensive guide to basic Git commands
Advanced R | Welcome	Deep dive into advanced R programming concepts
R Packages (2e)	Guide to creating and maintaining R packages
The Missing Semester of Your CS Education	Advanced topics within computer science
Grokking Algorithms	Illustrated guide to understanding algorithms
Scikit-Learn ML from Start to Finish | Kaggle	Comprehensive machine learning tutorial using Scikit-Learn
Another useful course on R	Useful for learning cluster analysis, GAMs, GLMs, etc

---

Fun Content

Name	Description
ThePrimeagen - YouTube	Content on learning Git and Vim motions
Theo - t3.gg - YouTube	Keep up to date on latest tech developments
Vim in VS Code - YouTube	Help you set up VIMmotions on VScode